

Scikit-learn

, often imported as `sklearn`, is a free and open-source Python library that has become the gold standard for traditional machine learning in Python . It is built on top of other essential scientific libraries like NumPy and SciPy and is designed to be accessible to everyone while remaining powerful enough for production use .

Its consistent and easy-to-use application programming interface (API) allows you to implement complex machine learning workflows quickly, making it an excellent tool for beginners and experts alike .

Here is a breakdown of what Scikit-learn offers.

Core Offerings and Capabilities

Scikit-learn provides a comprehensive suite of tools for almost every stage of a machine learning project . These can be grouped into several key areas:

- **Algorithms for Supervised Learning:** This includes a wide variety of algorithms for tasks where you have labeled data .
 - **Classification:** Identifying which category an object belongs to (e.g., spam detection, image recognition). Algorithms include Logistic Regression, Support Vector Machines (SVM), Random Forests, and Naive Bayes .
 - **Regression:** Predicting a continuous value (e.g., house prices, stock prices). Algorithms include Linear Regression, Ridge, Lasso, and Support Vector Regression (SVR) .
- **Algorithms for Unsupervised Learning:** For tasks where you have unlabeled data and want to find hidden patterns or structures .
 - **Clustering:** Grouping similar data points together (e.g., customer segmentation). Algorithms like K-Means and DBSCAN are available .
 - **Dimensionality Reduction:** Reducing the number of features in your data to combat the "curse of dimensionality" or for visualization (e.g., using Principal Component Analysis (PCA) or t-SNE) .
 - **Anomaly Detection:** Identifying unusual data points or outliers (e.g., fraud detection) .
- **Data Preprocessing:** Before you can feed data into a model, it often needs to be cleaned and transformed. Scikit-learn offers a suite of tools for this .
 - **Scaling:** Standardizing features (e.g., using `StandardScaler`).
 - **Encoding:** Converting categorical text data into numerical form (e.g., using `OneHotEncoder`).
 - **Imputation:** Filling in missing values (e.g., using `SimpleImputer`).
- **Model Selection and Evaluation:** It provides robust tools to help you choose and validate the best model for your data .

- **Model Evaluation:** A library of metrics to assess performance, such as accuracy, precision, recall, and F1-score for classification, and mean squared error (MSE) or R^2 for regression .
- **Cross-Validation:** Techniques to avoid overfitting by testing your model on different subsets of your data (e.g., using `cross_validate`) .
- **Hyperparameter Tuning:** Tools like `GridSearchCV` and `RandomizedSearchCV` to automatically find the best parameters for your chosen algorithm .
- **Pipeline Creation:** This is one of Scikit-learn's most powerful features. You can chain together a sequence of preprocessing steps and a final model into a single Pipeline object. This ensures reproducibility, avoids common mistakes like data leakage, and simplifies your code .

Key Benefits and Strengths

- **Consistent and Simple API:** The library is designed around a standard interface. All estimators (models) have a `.fit()` method for training and a `.predict()` method for making predictions. This makes switching between different algorithms very straightforward .
- **Excellent Documentation:** Scikit-learn is renowned for its high-quality, well-structured documentation with numerous examples .
- **Huge Community:** As an open-source project, it has a massive and active community of users and developers. This makes it reliable, ensures it's continuously updated, and means help is readily available online .
- **Integration:** It is designed to work seamlessly with the rest of the Python scientific ecosystem, including Pandas for data manipulation and Matplotlib for creating visualizations .

Here's a comprehensive **scikit-learn cheatsheet** covering the most common tasks, algorithms, and code snippets.

1. Basic Workflow

```
python

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.pipeline import Pipeline

# Split data

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
```

```
)
```

```
# Scale features
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
# Or use a pipeline
```

```
pipeline = Pipeline([
```

```
    ('scaler', StandardScaler()),
```

```
    ('classifier', LogisticRegression())
```

```
])
```

```
pipeline.fit(X_train, y_train)
```

2. Classification Algorithms

Algorithm	Import	Use Case
Logistic Regression	<code>from sklearn.linear_model import LogisticRegression</code>	Binary/multiclass, interpretable
KNN	<code>from sklearn.neighbors import KNeighborsClassifier</code>	Simple, small datasets
SVM	<code>from sklearn.svm import SVC</code>	High-dimensional, margin-based
Decision Tree	<code>from sklearn.tree import DecisionTreeClassifier</code>	Interpretable, non-linear
Random Forest	<code>from sklearn.ensemble import RandomForestClassifier</code>	High accuracy, robust
XGBoost	<code>from xgboost import XGBClassifier</code>	State-of-the-art for tabular
Naïve Bayes	<code>from sklearn.naive_bayes import GaussianNB</code>	Text, quick baseline
Neural Net	<code>from sklearn.neural_network import MLPClassifier</code>	Non-linear patterns

python

```
# Example

from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

3. Regression Algorithms

Algorithm	Import
Linear Regression	from sklearn.linear_model import LinearRegression
Ridge/Lasso	from sklearn.linear_model import Ridge, Lasso
ElasticNet	from sklearn.linear_model import ElasticNet
Decision Tree Regressor	from sklearn.tree import DecisionTreeRegressor
Random Forest Regressor	from sklearn.ensemble import RandomForestRegressor
SVR	from sklearn.svm import SVR
python	
	from sklearn.ensemble import RandomForestRegressor
	reg = RandomForestRegressor(n_estimators=100, random_state=42)
	reg.fit(X_train, y_train)
	y_pred = reg.predict(X_test)

4. Clustering

Algorithm	Import
K-Means	from sklearn.cluster import KMeans
DBSCAN	from sklearn.cluster import DBSCAN

Algorithm	Import
Hierarchical	from sklearn.cluster import AgglomerativeClustering
Gaussian Mixture	from sklearn.mixture import GaussianMixture

```
python
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=3, random_state=42)
labels = kmeans.fit_predict(X)
```

5. Dimensionality Reduction

Algorithm	Import
PCA	from sklearn.decomposition import PCA
t-SNE	from sklearn.manifold import TSNE
UMAP	import umap (external)
LDA (supervised)	from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

```
python
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

6. Model Evaluation

Classification Metrics

```
python
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix, classification_report
)
```

```
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, average='weighted')
rec = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')
auc = roc_auc_score(y_test, y_proba, multi_class='ovr')
cm = confusion_matrix(y_test, y_pred)
report = classification_report(y_test, y_pred)
```

Regression Metrics

```
python

from sklearn.metrics import (
    mean_squared_error, mean_absolute_error, r2_score
)

mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
rmse = mean_squared_error(y_test, y_pred, squared=False)
```

7. Cross-Validation

```
python

from sklearn.model_selection import cross_val_score, cross_validate

# Simple CV
scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
print(f"Mean: {scores.mean():.3f} (+/- {scores.std():.3f})")

# Multiple metrics
scoring = ['accuracy', 'precision_macro', 'recall_macro', 'f1_macro']
cv_results = cross_validate(model, X, y, cv=5, scoring=scoring, return_train_score=True)
```

8. Hyperparameter Tuning

Grid Search

python

```
from sklearn.model_selection import GridSearchCV
```

```
param_grid = {  
    'n_estimators': [50, 100, 200],  
    'max_depth': [5, 10, None],  
    'min_samples_split': [2, 5, 10]  
}
```

```
grid = GridSearchCV(  
    RandomForestClassifier(random_state=42),  
    param_grid,  
    cv=5,  
    scoring='f1_macro',  
    n_jobs=-1  
)
```

```
grid.fit(X_train, y_train)  
print(grid.best_params_)  
best_model = grid.best_estimator_
```

Random Search

python

```
from sklearn.model_selection import RandomizedSearchCV
```

```
from scipy.stats import randint, uniform
```

```
param_dist = {  
    'n_estimators': randint(50, 500),  
    'max_depth': randint(3, 20),  
    'min_samples_split': randint(2, 20)  
}
```

```
random_search = RandomizedSearchCV(  
    RandomForestClassifier(random_state=42),  
    param_distributions=param_dist,  
    n_iter=50,  
    cv=5,  
    random_state=42,  
    n_jobs=-1  
)  
random_search.fit(X_train, y_train)
```

9. Feature Selection

python

```
from sklearn.feature_selection import SelectKBest, f_classif, RFE, mutual_info_classif
```

Univariate selection

```
selector = SelectKBest(f_classif, k=10)
```

```
X_selected = selector.fit_transform(X, y)
```

RFE (Recursive Feature Elimination)

```
from sklearn.feature_selection import RFE
```

```
selector = RFE(estimator=LogisticRegression(), n_features_to_select=10)
```

```
X_selected = selector.fit_transform(X, y)
```

Feature importance from tree models

```
importances = model.feature_importances_
```

10. Preprocessing

Scaling

python

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler
```



```
# Standardization (mean=0, std=1)
```

```
scaler = StandardScaler()
```

```
# Min-max (range 0-1)
```

```
scaler = MinMaxScaler()
```

```
# Robust (uses median, IQR - handles outliers)
```

```
scaler = RobustScaler()
```

Encoding

```
python
```

```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
```

```
from sklearn.compose import ColumnTransformer
```

```
# Label encoding (for target)
```

```
le = LabelEncoder()
```

```
y_encoded = le.fit_transform(y)
```

```
# One-hot encoding (for features)
```

```
ohe = OneHotEncoder(sparse_output=False)
```

```
X_encoded = ohe.fit_transform(X_categorical)
```

```
# Column transformer
```

```
preprocessor = ColumnTransformer([  
    ('num', StandardScaler(), numerical_cols),  
    ('cat', OneHotEncoder(), categorical_cols)  
)
```

Imputation

```
python
```

```
from sklearn.impute import SimpleImputer
```

```
imputer = SimpleImputer(strategy='median') # or 'mean', 'most_frequent', 'constant'
X_imputed = imputer.fit_transform(X)
```

11. Common Pipelines

Full Classification Pipeline

python

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
```

Define preprocessor

```
preprocessor = ColumnTransformer([
    ('num', Pipeline([
        ('impute', SimpleImputer(strategy='median')),
        ('scale', StandardScaler())
    ]), numerical_cols),
    ('cat', Pipeline([
        ('impute', SimpleImputer(strategy='most_frequent')),
        ('encode', OneHotEncoder(handle_unknown='ignore'))
    ]), categorical_cols)
])
```

Full pipeline

```
pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', RandomForestClassifier(random_state=42))
])
```

```
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
```

12. Saving & Loading Models

```
python

import joblib

# Save

joblib.dump(model, 'model.pkl')
joblib.dump(pipeline, 'pipeline.pkl')

# Load

model = joblib.load('model.pkl')
pipeline = joblib.load('pipeline.pkl')

# Or using pickle

import pickle

with open('model.pkl', 'wb') as f:
    pickle.dump(model, f)

with open('model.pkl', 'rb') as f:
    model = pickle.load(f)
```

13. Quick Reference: Common Parameters

Parameter	Description
random_state	Set seed for reproducibility
n_jobs=-1	Use all CPU cores
verbose=1	Show progress
class_weight='balanced'	Handle class imbalance
shuffle=True	Shuffle data

Parameter	Description
n_estimators	Number of trees (RF, GBM)
learning_rate	Step size (GBM, XGBoost)
C	Inverse regularization strength (SVM, LR)
kernel	'linear', 'rbf', 'poly', 'sigmoid' (SVM)

14. Useful Utilities

python

Generate synthetic data

```
from sklearn.datasets import make_classification, make_regression, make_blobs
```

```
X, y = make_classification(n_samples=1000, n_features=20, n_informative=15, random_state=42)
```

Label binarization

```
from sklearn.preprocessing import label_binarize
```

```
y_bin = label_binarize(y, classes=[0, 1, 2])
```

Train-test split with stratification

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y)
```

Learning curve

```
from sklearn.model_selection import learning_curve
```

```
train_sizes, train_scores, test_scores = learning_curve(
    model, X, y, cv=5, train_sizes=np.linspace(0.1, 1.0, 10)
)
```

15. Commonly Used Scoring Metrics

Scoring	For
'accuracy'	Classification
'precision_macro'	Classification
'recall_macro'	Classification
'f1_macro'	Classification
'roc_auc_ovr'	Multi-class AUC
'neg_mean_squared_error'	Regression
'neg_mean_absolute_error'	Regression
'r2'	Regression
'adjusted_rand_score'	Clustering

16. Handling Imbalanced Data

python

```
from imblearn.over_sampling import SMOTE
```

```
from imblearn.pipeline import Pipeline as ImbPipeline
```

SMOTE

```
smote = SMOTE(random_state=42)
```

```
X_resampled, y_resampled = smote.fit_resample(X_train, y_train)
```

Pipeline with SMOTE

```
pipeline = ImbPipeline([
    ('scaler', StandardScaler()),
    ('smote', SMOTE(random_state=42)),
```

```
    ('classifier', RandomForestClassifier(random_state=42))  
])
```

17. Debugging & Inspection

python

Get feature names

```
feature_names = pipeline.named_steps['preprocessor'].get_feature_names_out()
```

Get coefficients (linear models)

```
coefs = model.coef_
```

Feature importance (tree models)

```
importances = model.feature_importances_
```

Partial dependence plots

```
from sklearn.inspection import PartialDependenceDisplay
```

```
PartialDependenceDisplay.from_estimator(model, X, features=[0, 1])
```